



Squaring a Sorted Array (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
 - Time complexity
 - Space complexity

Problem Statement

Given a sorted array, create a new array containing **squares of all the numbers of the input array** in the sorted order.

Example 1:

Input: [-2, -1, 0, 2, 3]
Output: [0, 1, 4, 4, 9]

Example 2:

Input: [-3, -1, 0, 1, 2]
Output: [0, 1, 1, 4, 9]

Try it yourself

Try solving this question here:



```
1 class SortedArraySquares {
2
3     public static int[] makeSquares(int[] arr) {
4         int[] square = new int[arr.length];
5         for(int val=0; val<arr.length; val++)
6             square[val] = (arr[val]) * (arr[val]);
7         int l=0,r=square.length-1;
8         while(r>l){
9             if(square[l]> square[r]) {
10                 int temp = square[r];
11                 square[r] = square[l];
12                 square[l] = temp;
13             }
14             r--;
```



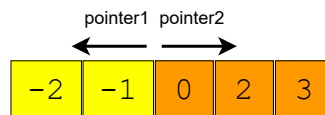
```
15     }  
16     return square;  
17 }  
18 }  
19
```

[Test](#)[Save](#)[Reset](#)

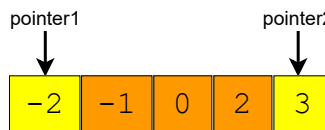
Solution

This is a straightforward question. The only trick is that we can have negative numbers in the input array, which will make it a bit difficult to generate the output array with squares in sorted order.

An easier approach could be to first find the index of the first non-negative number in the array. After that, we can use **Two Pointers** to iterate the array. One pointer will move forward to iterate the non-negative numbers, and the other pointer will move backward to iterate the negative numbers. At any step, whichever number gives us a bigger square will be added to the output array. For the above-mentioned Example-1, we will do something like this:



Since the numbers at both ends can give us the largest square, an alternate approach could be to use two pointers starting at both ends of the input array. At any step, whichever pointer gives us the bigger square, we add it to the result array and move to the next/previous number according to the pointer. For the above-mentioned Example-1, we will do something like this:



Code

Here is what our algorithm will look like:

[Java](#)[Python3](#)[C++](#)[JS](#)

```
1 class SortedArraySquares {  
2  
3     public static int[] makeSquares(int[] arr) {  
4         int n = arr.length;  
5         int[] squares = new int[n];  
6         int highestSquareIdx = n - 1;  
7         int left = 0, right = arr.length - 1;  
8         while (left <= right) {
```

[Tr](#)

```
9     int leftSquare = arr[left] * arr[left];
10    int rightSquare = arr[right] * arr[right];
11    if (leftSquare > rightSquare) {
12        squares[highestSquareIdx--] = leftSquare;
13        left++;
14    } else {
15        squares[highestSquareIdx--] = rightSquare;
16        right--;
17    }
18 }
19 return squares;
20 }
21
22 public static void main(String[] args) {
23
24     int[] result = SortedArraySquares.makeSquares(new int[] { -2, -1, 0, 2, 3 });
25     for (int num : result)
26         System.out.print(num + " ");
27     System.out.println();
28
29     result = SortedArraySquares.makeSquares(new int[] { -3, -1, 0, 1, 2 });
30     for (int num : result)
```

Run

Save

Reset



Time complexity

The above algorithm's time complexity will be $O(N)$ as we are iterating the input array only once.

Space complexity

The above algorithm's space complexity will also be $O(N)$; this space will be used for the output array.

[← Back](#)[✓ Completed](#)[Next →](#)[Remove Duplicates \(easy\)](#)[Triplet Sum to Zero \(medium\)](#)